

Advanced AI Vision Projects

These projects use:

- Python
 - OpenCV
 - YOLO
 - TensorFlow / PyTorch
 - Real-time camera processing
 - AI-based detection systems
-

1. Smart Helmet Detection System

Features

- Detect person
 - Detect helmet
 - Detect rider without helmet
 - Real-time alert
 - Save evidence image
 - Traffic AI project
-

Install

```
pip install ultralytics  
pip install opencv-python
```

Helmet Detection Code

```
from ultralytics import YOLO  
import cv2  
import os  
import time
```

```

# Load custom helmet model
model = YOLO("helmet.pt")

cap = cv2.VideoCapture(0)

save_path = "violations"

os.makedirs(save_path, exist_ok=True)

while True:

    ret, frame = cap.read()

    results = model(frame)

    annotated = results[0].plot()

    boxes = results[0].boxes

    for box in boxes:

        cls = int(box.cls[0])
        conf = float(box.conf[0])

        label = model.names[cls]

        x1, y1, x2, y2 = map(int, box.xyxy[0])

        if label == "no_helmet" and conf > 0.5:

            cv2.putText(
                annotated,
                "NO HELMET ALERT",
                (x1, y1 - 20),
                cv2.FONT_HERSHEY_SIMPLEX,
                1,
                (0,0,255),
                2
            )

            filename = f"{save_path}/{time.time()}.jpg"
            cv2.imwrite(filename, frame)

            print("Violation Saved")

```

```
cv2.imshow("Helmet Detection", annotated)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

2. Vehicle Detection System

Features

- Car detection
 - Bike detection
 - Bus detection
 - Truck detection
 - Vehicle counting
 - Smart traffic monitoring
-

Vehicle Detection Code

```
from ultralytics import YOLO
import cv2

model = YOLO("yolov8n.pt")

cap = cv2.VideoCapture("traffic.mp4")

vehicle_classes = ["car", "motorcycle", "bus", "truck"]

count = 0

while True:

    ret, frame = cap.read()

    if not ret:
```

```

        break

results = model(frame)

boxes = results[0].boxes

for box in boxes:

    cls = int(box.cls[0])
    label = model.names[cls]

    if label in vehicle_classes:

        count += 1

        x1, y1, x2, y2 = map(int, box.xyxy[0])

        cv2.rectangle(frame, (x1,y1), (x2,y2), (0,255,0), 2)

        cv2.putText(
            frame,
            label,
            (x1, y1-10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.7,
            (0,255,0),
            2
        )

cv2.putText(
    frame,
    f"Vehicles: {count}",
    (20,50),
    cv2.FONT_HERSHEY_SIMPLEX,
    1,
    (255,0,0),
    2
)

cv2.imshow("Vehicle Detection", frame)

if cv2.waitKey(1) == ord('q'):
    break

```

```
cap.release()
cv2.destroyAllWindows()
```

3. Fire and Smoke Detection System

Features

- Detect fire
 - Detect smoke
 - Emergency alarm
 - Industrial safety AI
 - Forest fire detection
-

Fire Detection Code

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    lower_fire = np.array([0, 120, 150])
    upper_fire = np.array([35, 255, 255])

    mask = cv2.inRange(hsv, lower_fire, upper_fire)

    contours, _ = cv2.findContours(
        mask,
        cv2.RETR_TREE,
        cv2.CHAIN_APPROX_SIMPLE
    )

    for contour in contours:
```

```
area = cv2.contourArea(contour)

if area > 3000:

    x,y,w,h = cv2.boundingRect(contour)

    cv2.rectangle(frame, (x,y), (x+w,y+h), (0,0,255), 2)

    cv2.putText(
        frame,
        "FIRE DETECTED",
        (x, y-10),
        cv2.FONT_HERSHEY_SIMPLEX,
        1,
        (0,0,255),
        2
    )

    print("ALERT: FIRE DETECTED")

cv2.imshow("Fire Detection", frame)

if cv2.waitKey(1) == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

4. AI Drone Vision System

Features

- Real-time aerial detection
 - Object tracking
 - Human detection
 - Vehicle tracking
 - Autonomous drone AI
-

Drone Vision Code

```
from ultralytics import YOLO
import cv2

model = YOLO("yolov8n.pt")

cap = cv2.VideoCapture("drone_video.mp4")

while True:

    ret, frame = cap.read()

    if not ret:
        break

    results = model.track(frame, persist=True)

    annotated = results[0].plot()

    cv2.imshow("Drone AI Vision", annotated)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

5. Thermal AI Camera System

Features

- Human heat detection
 - Temperature monitoring
 - Thermal anomaly detection
 - Security AI
 - Industrial thermal inspection
-

Thermal Camera AI Code

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    thermal = cv2.applyColorMap(gray, cv2.COLORMAP_JET)

    cv2.imshow("Thermal Camera", thermal)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Advanced Upgrade Ideas

Smart Helmet Detection

- Number plate OCR
- Rider tracking
- Traffic database
- Auto challan system

Vehicle Detection

- Speed estimation
- Traffic density AI
- Smart signals
- Accident detection

Fire Detection

- Smoke classification
- IoT sprinkler activation
- GSM emergency alert

Drone Vision

- GPS tracking
- Autonomous navigation
- AI target lock
- Defense surveillance

Thermal Camera

- MLX90640 integration
 - ESP32 streaming
 - Human temperature AI
 - Thermal anomaly alerts
-

Best Hardware

AI Edge Devices

- NVIDIA Jetson Nano
 - Raspberry Pi Foundation Raspberry Pi 5
 - ESP32-CAM
 - MLX90640 Thermal Sensor
 - USB AI Cameras
-

Best Libraries

- OpenCV
- TensorFlow
- PyTorch

- YOLO
 - NumPy
 - MediaPipe
-

Future Advanced Projects

1. AI Border Security
2. AI Military Surveillance
3. Autonomous UAV Swarm
4. AI Smart City Monitoring
5. Industrial AI Inspection
6. AI Defense Robot
7. Wildlife AI Monitoring
8. Smart Railway AI
9. AI Forest Monitoring
10. AI Disaster Detection